CS331: COMPUTER NETWORKS PROJECT

Scalable Network I/O: From select to io_uring

An Empirical & Architectural Analysis of Linux Socket Multiplexing

Suchith
Architecture Lead & io_uring

Rohith
epoll (ET) & Sockets

Harshith
select & poll Engines

Hanook
Benchmarking

1. Problem Statement & Objectives

Modern high-concurrency network servers quickly break down under the **C10K Concurrency Barrier** when using traditional paradigms. We set out to solve three core bottlenecks:

- **Thread Stack Exhaustion:** Traditional blocking servers allocate ~8MB per thread. 1,000 connections instantly consume gigabytes of RAM.
- **Context Switch Thrashing:** The CPU wastes over 90% of its cycles swapping registers and invalidating caches instead of serving requests.
- **O(N) Syscall Overhead:** Early multiplexers force the kernel to scan every single socket descriptor per event, devastating throughput.



Objective: Systematically build, profile, and compare 5 distinct Linux TCP socket paradigms from scratch under extreme load (up to 5,000+ connections).

2. Architecture & Mechanism

To understand the paradigm shift, we engineered five standalone C TCP servers, moving from synchronous readiness polling to zero-syscall asynchronous completion rings.

1. Blocking I/O

Thread-per-client model. Spawns via pthread_create(). Incurs massive memory and scheduling overhead.

2. select & poll

Synchronous readiness. Iterates arrays in **O(N)** time. select hits a strict 1024 FD cap.

3. epoll (ET)

Uses kernel Red-Black Trees for O(1) ready lists. Implemented Level-Triggered (LT) and Edge-Triggered (ET) with short-read buffer draining.

4. io_uring

Asynchronous Completion Ring. Optimized with multishot receive and provided buffer rings (IOSQE_BUFFER_SELECT) for zero-syscall fast path.

3. Extension, Issues Fixed & Evaluation

🛠️ Harness & Verification Built

We built custom multi-threaded C load testing tools extending the server codebase for rigorous empirical validation:

- **Automated Test Suite:** Verified 29-byte echo correctness across all 10 server variants via test_servers.py and test_new_loadgen.py (100% pass rate).
- **Client Bottleneck Diagnosed:** Proved Python asyncio client was 99% saturated (102% of one core), starving server core 0 at only ~19% CPU.
- **Native C Load Generator:** Built loadgen.c with pinned worker threads & per-thread epoll, providing >50% headroom while driving server core to 98.7% CPU.

📈 Empirical Evaluation Results

Under saturated server core load (98.7% CPU), true architectural scaling was uncovered:

- **Peak Concurrency:** io_uring delivered peak 234,585 req/s at 100 conns. epoll_lt achieved 201,676 req/s at 1,000 conns.
- **Poll Reversal Disproven:** With unbottlenecked client, epoll beat poll across all active scales (epoll 201.7k vs poll 104.4k at 1k conns — 1.93x advantage).
- **ET Superiority at Scale:** At 5,000 active burst connections, epoll_et sustained 146,287 req/s with 12% lower CPU cost (7.54 us/req) and 24% fewer CS than LT.

4. Non-Functional Testing Parameters

Empirical performance, memory footprint (User VmRSS + Kernel TCP), and limits across 5,000 persistent connections.

I/O Paradigm	Throughput (5k Conns)	Scalability (Memory Footprint)	Reliability & CPU Cost
Blocking	53,807 req/s	Poor (33.1 MB RSS / 19.5 GB virt)	High crash risk; 17.29 us/req CPU; high CS
select()	128,797 req/s (1k)	Excellent (1.5 MB RSS)	Fails strictly at 1024 FDs (FD_SETSIZE cap)
poll()	113,883 req/s (104k 1k)	Excellent (1.6 MB RSS)	O(N) pollfd scan; 9.81 us/req CPU strain
epoll (ET/LT)	146,287 req/s (Peak 219k)	Best (1.78 MB lean RSS)	Production standard; 7.54 us/req; lowest latency
io_uring	234,585 req/s (Peak 100)	Constant (36.1 MB pool)	Zero syscalls in fast path; CQ lock pressure at 5k

5. Key Technical Challenges Faced

- The select() FD_SETSIZE Hard Cap: Our initial `select` server crashed violently when load testing exceeded 1024 connections. We had to implement an explicit guard check if (client_fd >= FD_SETSIZE) to cleanly reject overflow connections and prevent stack corruption.

- Draining Edge-Triggered Buffers & Wasted Syscalls: Standard ET epoll drains with read() until EAGAIN, wasting a 3rd syscall per request. We implemented short-read detection (read < BUF_SIZE) to stop draining immediately on drained stream sockets, eliminating wasted syscalls while preventing busy-spinning.

- io_uring Multishot Recv & Buffer Rings: Upgraded from single-shot to multishot receive (io_uring_prep_recv_multishot) combined with provided buffer rings (IOSQE_BUFFER_SELECT). This eliminated per-connection 4KB buffers, resolved ENOBUFS, and reduced syscall overhead to 0.08 per request.



6. Scenario 2: The C10K Idle Connection Sweep

Evaluating C10K Scalability: 100 Active Connections with 0 to 10,000 Idle Connections

The O(N) Collapse of poll()

- **Monotonic Degradation:** poll() throughput plummets from 59,833 req/s (0 idle) to 15,734 req/s (10,000 idle) — a severe 73.7% throughput collapse.
- **Algorithmic Bottleneck:** Every poll() iteration forces the Linux kernel to copy and linearly scan 10,100 struct pollfd elements across user/kernel boundary.
- **Tail Latency Ballooning:** p99 latency nearly quadruples from 4.2 ms to 15.9 ms under 10k idle connections due to CPU exhaustion scanning inactive sockets.
- **No Active Crossover:** With the fast C client, epoll and io_uring beat poll starting at 0 idle connections; poll never wins at any point in the idle sweep.

O(1) Kernel Scalability

- **Complete Immunity:** Both epoll and io_uring remain completely flat and impervious to 10,000 idle connections, sustaining ~255,000 to 293,000 req/s.
- **18x Throughput Advantage:** At 10,000 idle connections, epoll_et (255.7k req/s) and io_uring (292.9k req/s) outperform poll by more than 18.6x.
- **Sub-Millisecond Tail Latency:** epoll_et and io_uring maintain ultra-tight p99 latency (1.04 ms to 1.28 ms) even with 10,000 idle sockets connected.
- **Core Takeaway:** epoll readiness lists and io_uring completion rings scale strictly with active traffic O(K), solving the C10K problem completely.

7. Key Conclusions & Methodological Lessons

Empirical Takeaways from Benchmarking Linux High-Performance Network Sockets

Architecture & Memory Audit

• **epoll vs io_uring Sweet Spots:** io_uring wins at low-to-medium scale (234.6k req/s at 100 conns) via zero syscalls. epoll ET wins at 5k active bursts (146.3k req/s) due to lock-free execution without ring contention.

• **Level-Triggered vs Edge-Triggered:** LT edges out ET at 100-1k conns (+16% to +35%) by avoiding loop checks. ET wins at 5k conns (+10.3% req/s, -12% CPU) by eliminating redundant kernel event notifications.

• **Memory Footprint Demystified:** epoll RSS was slashed from 21.6 MB to 1.78 MB by replacing flat static arrays with sparse pointer tables. io_uring requires fixed 36 MB for its 8k-entry buffer pool.

• **Kernel TCP Socket Overhead:** Measured via /proc/net/sockstat: Linux kernel TCP buffer memory scales strictly at ~7.17 KB per connection across all architectures.

Benchmarking Methodology

• **The Multi-Core Client Fallacy:** Never rely on core-averaged CPU utilization (e.g. mpstat 6.8% across 16 cores). An interpreted single-threaded client will saturate at 100% CPU on one core while masking starvation.

• **The Artificial Batching Trap:** A slow client causes packet backlog in socket queues. This artificially aids clumsy O(N) multiplexers (poll) while forcing efficient engines (epoll) into 180k+ context switches.

• **Closed-Loop Saturation Calibration:** Always calibrate load generators against a reference server to guarantee the server core reaches >90% CPU while client threads retain >40% headroom.

• **Beyond Throughput:** Accurate systems evaluation requires measuring per-thread CPU utilization, voluntary/involuntary context switches, wait batch ratios, and kernel socket memory.